5. Компютърни архитектури. Формати на данните. ЦПУ

Тема: 5. Компютърни архитектури. Формати на данните. ЦПУ

Компютърната архитектура описва логическата организация на компютърната система, начина, по който са свързани основните ѝ компоненти, начина на представяне на данните и принципите, по които процесорът изпълнява инструкции. Тази тема обединява три основни въпроса: как е устроен компютърът като система, как се представят различните видове данни в двоичен вид и как централният процесор обработва тези данни чрез инструкции. Темата е част от официалния конспект за държавния изпит по „Софтуерно инженерство“.

Основополагащ принцип в съвременните компютри е принципът на запомнената програма. Според него както данните, така и инструкциите на програмата се съхраняват в една и съща памет. Това е съществена идея, защото позволява програмата да бъде разглеждана като последователност от данни, които процесорът извлича, декодира и изпълнява. Благодарение на този принцип една и съща машина може да изпълнява различни програми, без да се променя хардуерът ѝ. Достатъчно е в паметта да бъде заредена друга последователност от инструкции.

Общата структура на компютъра включва централен процесор, оперативна памет, входно-изходни устройства и системни връзки между тях. Централният процесор изпълнява инструкциите на програмата и управлява обработката на данните. Оперативната памет съхранява инструкциите и данните, които се използват по време на изпълнение. Входно-изходните устройства осигуряват връзката на компютъра с външната среда, например клавиатура, дисплей, диск, мрежова карта и други устройства. Между тези компоненти има шини или други комуникационни механизми, чрез които се предават адреси, данни и управляващи сигнали.

Концептуалното изпълнение на една инструкция може да се разгледа като цикъл, който се повтаря за всяка инструкция от програмата. Първо процесорът трябва да знае откъде да вземе следващата инструкция. За това служи програмният брояч, който съдържа адреса на следващата инструкция в паметта. След това инструкцията се извлича от паметта и се записва в регистъра на инструкциите. След извличането тя се декодира, тоест процесорът определя какъв е кодът на

операцията, какви операнди са нужни и къде се намират те. След декодирането се извличат операндите, ако инструкцията изисква такива. След това се изпълнява операцията, например събиране, сравнение, запис в паметта или преход. Накрая резултатът се записва в регистър или в паметта, а програмният брояч се обновява така, че да сочи следващата инструкция. При обикновена последователност той се увеличава, а при преход може да получи нова стойност.

Всички данни в компютъра се представят чрез битове. Битът е най-малката единица информация и може да има стойност нула или единица. От битове се изграждат по-големи единици, например байтове, думи и двойни думи. Смисълът на една двоична последователност зависи от формата, в който я интерпретираме. Една и съща последователност от битове може да бъде цяло число, символ, машинна инструкция или част от число с плаваща запетая. Затова форматите на данните са съществена част от компютърната архитектура.

Целите двоични числа са основен формат за представяне на дискретни числови стойности. При беззнаковите цели числа всички битове участват в представянето на абсолютната стойност. Ако числото се представя с n бита, възможните стойности са от 0 до $2^n - 1$. Например с осем бита могат да се представят беззнакови стойности от 0 до 255. Всеки бит има тегло, което е степен на двойката, а стойността на числото се получава като сума от теглата на битовете със стойност единица.

Когато трябва да се представят и отрицателни числа, е необходим формат със знак. Исторически съществуват различни подходи, например знак и абсолютна стойност, обратен код и допълнителен код до две. Най-важният и най-широко използван е допълнителният код до две. При него най-старшият бит може да се разглежда като знаков, но представянето не е просто отделен знак и стойност. Положителните числа се представят както при обикновеното двоично представяне. Отрицателното число се получава, като се вземе двоичното представяне на съответната положителна стойност, обърнат се всички битове и се добави единица.

Допълнителният код до две е важен, защото позволява аритметичните операции събиране и изваждане да се реализират по един и същ хардуерен начин както за положителни, така и за отрицателни числа. Например изваждането може да се реализира като събиране с отрицателното число. Освен това в този формат има само едно представяне на нулата, което е предимство спрямо някои други формати. Диапазонът на n-битово число в допълнителен код до две е от -2^{n-1} до $2^{n-1}-1$. Тази асиметрия възниква, защото нулата заема едно от неотрицателните представяния.

При операции с цели числа е важно да се разграничават пренос и препълване. Преносът е свързан с беззнакова аритметика и показва, че резултатът е излязъл извън наличния брой битове. Препълването е свързано със знакова аритметика и означава, че полученият резултат не може да бъде представен коректно в зададения формат със знак. Например при събиране на две положителни числа, ако резултатът стане отрицателен според знаковия бит, имаме препълване. Тези

състояния се отразяват чрез флагове в регистъра на състоянието.

Двоично-десетичните числа представляват формат, при който всяка десетична цифра се кодира отделно чрез четири бита. Този формат е известен като BCD, или binary-coded decimal. Например десетичното число 59 се представя чрез две групи от по четири бита: едната за цифрата 5, а другата за цифрата 9. Предимството на този формат е, че се запазва точната десетична структура на числото, което е удобно при финансови и счетоводни изчисления, където закръгленията в двоична плаваща запетая могат да бъдат нежелани. Недостатъкът е, че форматът е неикономичен, защото от шестнадесет възможни комбинации на четири бита се използват само десет.

Числата с плаваща запетая служат за представяне на реални числа и числа с много голям или много малък порядък. Те се основават на идея, подобна на научния запис на числата. Числото се представя чрез знак, мантиса и експонента. Общата идея е стойността да бъде от вида $(-1)^s \cdot m \cdot 2^e$, където s определя знака, m е мантисата, а e е експонентата. Мантисата съдържа значещите битове, а експонентата определя позицията на двоичната запетая.

При числа с плаваща запетая има компромис между диапазон и точност. Експонентата позволява да се представят много големи и много малки числа, но мантисата има ограничен брой битове, затова не всички реални числа могат да бъдат представени точно. В резултат възникват грешки от закръгляне. Това означава, че операции като събиране и изваждане с плаваща запетая не винаги се държат като идеалната реална аритметика. Например малки грешки могат да се натрупват, а сравняването на две числа с плаваща запетая за точно равенство често е проблематично.

В стандартното представяне на числа с плаваща запетая, например IEEE 754, експонентата обикновено се съхранява с отместване, наречено bias. Това позволява експоненти с различен знак да бъдат представени като беззнакови стойности. Освен нормализирани числа се поддържат и специални стойности, като нула, безкрайност и NaN, което означава „not a number“. Тези специални стойности са необходими за обработка на гранични случаи, например деление на нула или неопределени операции.

Символните данни се представят чрез кодови таблици. Кодовата таблица съпоставя на всеки символ числова стойност. ASCII е класически пример за такава таблица. Тя използва седем бита за основните символи на английската азбука, цифри, пунктуация и управляващи символи. Разширени варианти използват осем бита. Ограничението на ASCII е, че не може да представя удобно символи от различни езици и писменост.

Unicode решава този проблем, като дефинира единно множество от кодови точки за символите на различни езици и системи за писане. Unicode не е само една таблица от байтове, а стандарт за кодови точки и начини за тяхното кодиране. UTF-8 е най-разпространеното кодиране, защото е съвместимо с ASCII за основните латински символи и използва променлив брой байтове за различните

символи. Това позволява ефективно представяне на текстове, съдържащи символи от различни езици.

Централният процесор е основният изпълнителен компонент на компютъра. Той реализира машинния език на архитектурата, тоест множеството инструкции, които компютърът може да изпълнява. Вътрешната структура на процесора включва регистри, аритметико-логическо устройство, регистър на състоянието, блок за управление и вътрешни връзки между тях.

Регистрите са малки, много бързи памети вътре в процесора. Те се използват за временно съхранение на операнди, резултати, адреси и управляваща информация. Достъпът до регистър е много по-бърз от достъпа до оперативната памет, затова процесорът се стреми да извършва операциите си върху данни, намиращи се в регистрите. Съществуват регистри с общо предназначение и специализирани регистри. Регистри с общо предназначение могат да участват в различни операции, докато специализираните имат конкретна роля.

Програмният брояч е специален регистър, който съдържа адреса на следващата инструкция. Регистърът на инструкциите съдържа текущо изпълняваната или декодирана инструкция. Регистърът на стека, или указателят към стека, сочи върха на стека и се използва при извикване на подпрограми, връщане от тях, съхраняване на локални данни и обработка на прекъсвания. Регистри могат да се използват и като базови или индексни регистри при адресиране на паметта.

Аритметико-логическото устройство, или ALU, е блокът, който извършва основните операции върху данните. То изпълнява аритметични операции като събиране и изваждане, логически операции като И, ИЛИ, изключващо ИЛИ и отрицание, операции за изместване и сравнение. Сравнението често се реализира чрез изваждане, при което резултатът не задължително се записва, но флаговете се обновяват. По този начин последваща условна инструкция може да реши дали да се извърши преход.

Регистърът на състоянието съдържа флагове, които описват свойствата на последния резултат или текущото състояние на процесора. Нулевият флаг показва дали резултатът е нула. Знаковият флаг показва знака на резултата при интерпретация със знак. Флагът за пренос показва пренос при беззнакова аритметика. Флагът за препълване показва некоректен резултат при знакова аритметика. Може да има и други флагове, например флаг за четност, флаг за посока при низови операции и флаг за разрешаване на прекъсвания. Флаговете са особено важни при условните преходи, защото позволяват програмата да реагира на резултатите от предходни операции.

Блокът за управление координира работата на процесора. Той декодира инструкциите и генерира управляващи сигнали към регистрите, аритметико-логическото устройство, паметта и другите блокове. В зависимост от архитектурата блокът за управление може да бъде реализиран твърдо логически или микропрограмно. При твърдата логика управляващите сигнали се генерират директно от комбинационни и последователностни схеми. При

микропрограмното управление всяка машинна инструкция се реализира чрез последователност от по-прости микроинструкции.

Конвейерната обработка е важна техника за ускоряване на изпълнението на инструкции. Идеята е изпълнението на инструкция да се раздели на етапи, като извличане, декодиране, извличане на операнди, изпълнение, достъп до памет и запис на резултата. Докато една инструкция се изпълнява в един етап, друга инструкция може да бъде в предходен или следващ етап. Така процесорът може да започва нова инструкция преди предишната да е завършила напълно.

Конвейерът увеличава пропускателната способност, но не премахва времето за изпълнение на отделната инструкция. Ако имаме конвейер с няколко етапа, идеалният случай е след запълването му да завършва по една инструкция на такт. На практика това не винаги е възможно, защото възникват конфликти. Структурни конфликти възникват, когато две инструкции имат нужда от един и същ хардуерен ресурс едновременно. Зависимости по данни възникват, когато една инструкция има нужда от резултат, който предходна инструкция още не е записала. Управляващи зависимости възникват при условни преходи, защото процесорът не знае предварително коя ще бъде следващата инструкция. За справяне с тези проблеми се използват техники като спиране на конвейера, препращане на резултати, предсказване на преходи и други.

Инструкцията на процесора е двоично кодирана команда, която задава операция и операнди. Форматът на инструкцията зависи от конкретната архитектура, но обикновено включва код на операцията, информация за операндите и евентуално префикси или допълнителни полета. Кодът на операцията определя какво трябва да се извърши: събиране, преместване, сравнение, преход, достъп до памет и т.н. Останалите полета указват къде са операндите и къде да се запише резултатът.

Префиксите са допълнителни байтове или полета, които променят интерпретацията на инструкцията. Те могат да задават размер на операнда, размер на адреса, използване на определен сегмент, повторение на низова инструкция или други модификации. Не всяка архитектура използва префикси по един и същ начин, но общата идея е, че префиксът не е самостоятелна операция, а изменя следващата инструкция.

Операндите на инструкцията могат да бъдат разположени в регистри, в паметта или в самата инструкция. Когато операндът е непосредствена стойност, той е записан като част от инструкцията. Когато е регистров операнд, инструкцията указва кой регистър да се използва. Когато операндът е в паметта, инструкцията трябва да съдържа или да позволява изчисляване на адреса му. Именно тук са важни моделите на адресация.

При непосредствена адресация стойността на операнда е записана директно в инструкцията. Това е удобно за константи. При регистрова адресация операндът се намира в регистър, което е бързо. При директна адресация инструкцията съдържа адрес на операнда в паметта. При индиректна адресация инструкцията указва място, където се намира адресът на операнда, например регистър, съдържащ адрес. При базова адресация ефективният адрес се получава чрез

базов регистър и отместване. При индексна адресация се използва индексен регистър, което е удобно за масиви. При базово-индексна адресация се комбинират базов регистър, индексен регистър, коефициент на мащабиране и отместване. Това позволява ефективно адресиране на структури от данни, масиви и стекови рамки.

Аритметико-логическите инструкции извършват изчисления и логически преобразувания. Те включват събиране, изваждане, умножение, деление, инкрементиране, декрементиране, логическо И, логическо ИЛИ, изключващо ИЛИ, побитово отрицание, измествания и ротации. Тези инструкции не само дават резултат, но често променят флаговете. Например инструкция за сравнение може да не запише резултат, но да обнови нулевия, знаковия, преносния и overflow флага, така че следваща условна инструкция да използва тази информация.

Низовите инструкции работят върху последователности от байтове, думи или други елементи. Те могат да копират, сравняват, търсят или записват последователни елементи в паметта. Обикновено използват специални регистри като указатели към източник и получател, както и брояч. След всяка операция адресите могат автоматично да се увеличават или намаляват според флаг за посока. Низовите инструкции са полезни при обработка на масиви, символни низове и блокове памет.

Инструкциите за преход променят нормалния последователен ход на изпълнение. Безусловният преход задава нова стойност на програмния брояч независимо от условия. Условният преход се изпълнява само ако определено условие, обикновено изразено чрез флаговете, е вярно. Например след сравнение може да се изпълни преход при равенство, при неравенство, при по-малко, при по-голямо, при пренос или при препълване. Чрез условни преходи се реализират разклонения, цикли и логика на управление.

Управлението на програмата включва не само преходи, но и извикване на подпрограми и връщане от тях. При извикване на подпрограма процесорът трябва да запомни адреса, на който трябва да се върне след края ѝ. Обикновено този адрес се записва в стека. След това програмният брояч се променя така, че да сочи началото на подпрограмата. При връщане адресът се извлича от стека и изпълнението продължава от мястото след извикването. Стекът може да съдържа също параметри, локални променливи и запазени регистри.

Важно е да се разбира разликата между архитектура на набора от инструкции и микроархитектура. Архитектурата на набора от инструкции описва какво вижда програмистът или компилаторът: регистри, инструкции, формати на данни, адресационни режими и поведение на програмата. Микроархитектурата описва как тези инструкции са реализирани вътрешно: конвейери, кешове, блокове за изпълнение, предсказване на преходи и други техники. Две машини могат да имат една и съща архитектура на инструкциите, но различна вътрешна реализация.

В заключение, компютърната архитектура свързва представянето на данните, организацията на процесора и изпълнението на инструкциите. Данните се

представят чрез двоични формати: цели числа, двоично-десетични числа, числа с плаваща запетая и символни кодове. Централният процесор съдържа регистри, аритметико-логическо устройство, регистър на състоянието и блок за управление. Инструкциите определят операциите, операндите и начините за адресиране. Конвейерната обработка позволява по-висока производителност чрез припокриване на етапите на изпълнение. Всички тези елементи заедно обясняват как една програма, записана в паметта, се превръща в последователност от конкретни хардуерни действия.